

Softwarové inženýrství

Beránek Pavel, 2024

Obsah seminářů

1. Proces a metodiky vývoje softwaru	21.2.2024	<i>Beránek P.</i>
2. Inženýrství požadavků	28.2.2024	<i>Beránek P.</i>
3. Softwarová architektura	6.3.2024	<i>Beránek P.</i>
4. Nástroje pro kontrolu verzí 1	13.3.2024	<i>Hřebejk P.</i>
5. Nástroje pro kontrolu verzí 2	20.3.2024	<i>Hřebejk P.</i>
6. Testování softwaru	27.3.2024	<i>Maškov V.</i>
7. Problematika API	10.4.2024	<i>Hřebejk P.</i>
8. Evoluce softwaru	17.4.2024	<i>Beránek P.</i>
9. Spolehlivost softwaru	24.4.2024	<i>Maškov V.</i>
10. Nasazení a podpora softwaru	15.5.2024	<i>Svoboda J.</i>

2x odpadnutí kvůli státním svátkům, 1x odpadnutí kvůli sportovnímu dni

Podmínky úspěšného splnění

- Zápočet

- ~~○ Praktická úloha v nástroji Git~~

- Odevzdání klasifikované seminární práce v textové podobě
 - Nabídka témat: [ZDE](#)
 - Téma může řešit jednotlivec nebo skupina maximálně o 3 studentech

- Zkouška

- Prezentace odevzdané klasifikované seminární práce a diskuze nad ní.
 - Kritéria hodnocení:
 - Počet řešitelů skupiny
 - Kvalita řešení teoretických východisek z vědeckých článků a učebnic (stačí 5 zdrojů)
 - Kvalita výzkumné metodiky
 - Kvalita výstupů (softwarový prototyp, diagramy, grafy)
 - Kvalita přednesu prezentace
 - Kvalita odevzdané textové části seminární práce
 - Schopnost reagovat na otázky z obecnstva
 - Dodržení citační normy APA, typografických pravidel a pravopisu českého jazyka

Seminář 1

Proces a metodiky
vývoje softwaru

<https://github.com/pavelberanek91/UJEP/tree/main/SWI/sem1>



1.1. Výroba, vývoj a tvorba

- Inženýrství

- disciplína, která se zabývá metodickým postupem jak navrhovat a implementovat produkt v požadované kvalitě

- Softwarové inženýrství

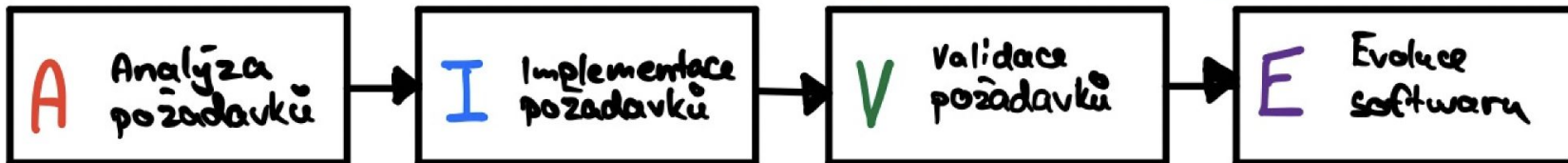
- jak vytvářet (nebo upravovat) takový software, aby byl dependabilní.
- Metodický postup, tedy návod jak postupovat, nazýváme procesem vývojem softwaru,

- Formy kreativní lidské činnosti

- **Výroba** - pokud se dodrží metodika, tak bude pokaždé produkt zcela stejný (výroba automobilu na sériové lince).
- **Tvorba**- metodika se nedodrжуje (nebo zanedbatelně), jelikož cílem je vytvořit pokaždé unikátní produkt (umělec maluje obraz).
- **Vývoj** - metodika se dodrжуje, ale je nutné ji upravit (nastavit parametry) a často improvizovat, tudíž produkt nebude nikdy stejný, ale bude často podobný.

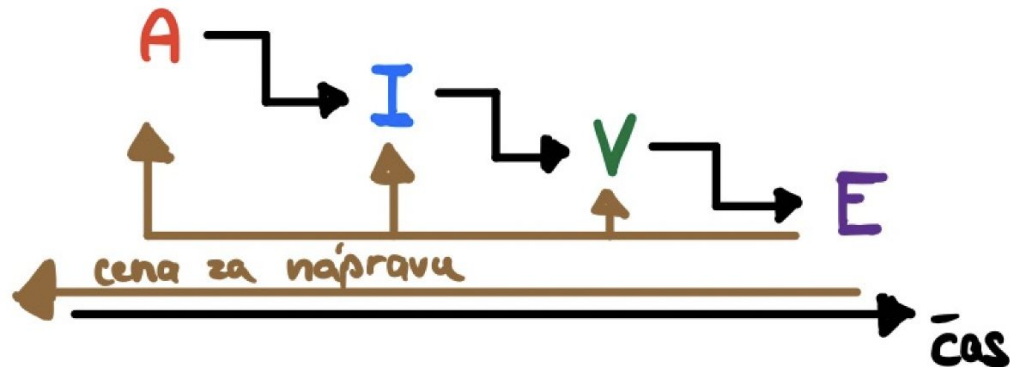
1.2. Obecný model procesu vývoje softwaru

- Proces vývoje softwaru
 - Proces je sekvence aktivit, aktivita je nějaká činnost/fáze (může také být procesem)
- Fáze procesu vývoje softwaru
 - Analýza požadavků
 - Zjišťování toho, co se má vyvíjet.
 - Implementace požadavků
 - Navrhování a programování zjištěných požadavků.
 - Validace požadavků
 - Ověření toho, že vyvinutý program plní zjištěné požadavky.
 - Evoluce požadavků
 - Přidání nových nebo aktualizace existujících požadavků na program.

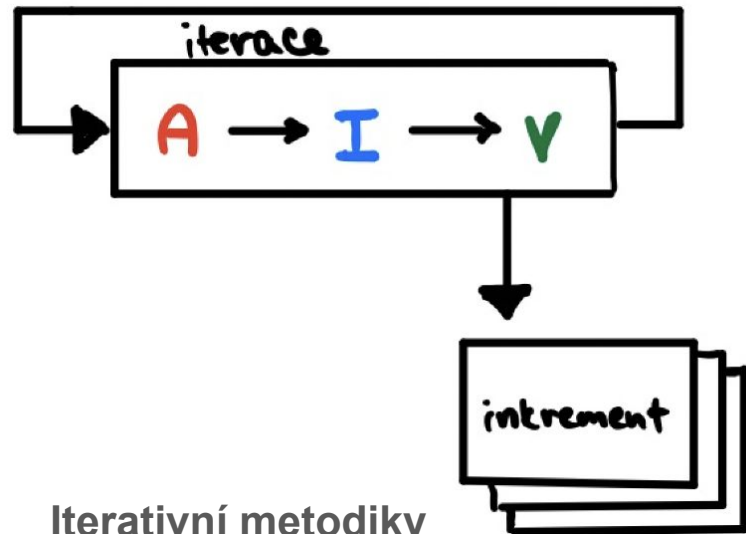


1.3. Typy modelů procesu vývoje softwaru

- Sekvenční metodiky
 - Vodopádová metodika, V-model, Rational Unified Process (RUP), Pomalé programování
- Iterativní metodiky
 - Rychlý vývoj aplikací (RAD), Boehmův spirálová metodika, Extrémní programování (XP), Scrum



Sekvenční metodiky



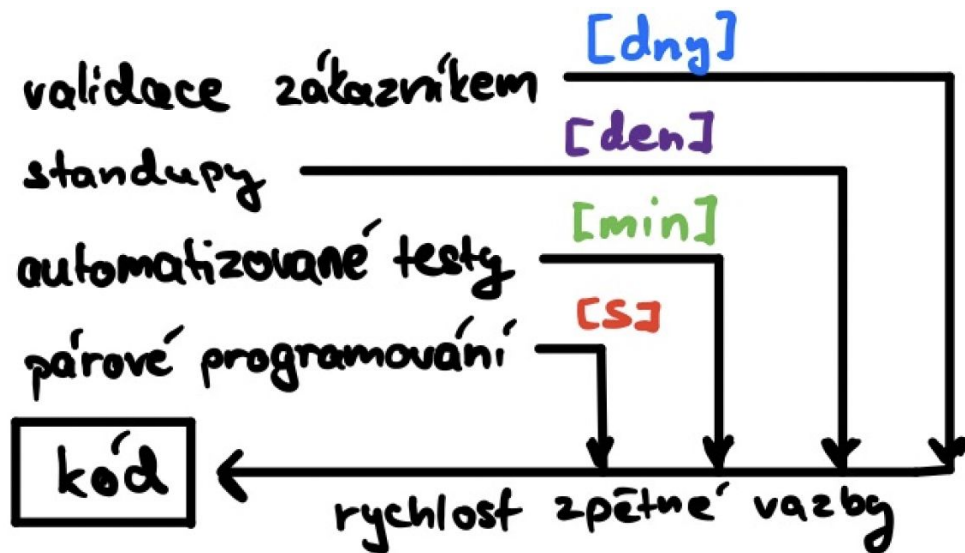
Iterativní metodiky

1.4. Agilní metodiky

- Agilní manifest
 - Deklaruje filosofii metodiky vývoje. Takové metodiky pak nazýváme agilní.
 - Převážná část agilních metodik jsou iterativního charakteru.
- Jakkoliv jsou **body napravo** hodnotné, **bodů nalevo** si ceníme více.
 - **jednotlivci a interakce** mají přednost před **procesy a nástroji**
 - **fungují software** má přednost před **obsáhlou dokumentací**
 - **spolupráce se zákazníkem** má přednost před **smluvním vyjednáváním**
 - **reagování na změny** má přednost před **postupováním podle plánu**

1.5. Extrémní programování

- Extrémní programování (XP)
 - Metodika, založená na zkracování zpětnovazebních smyček.
- Techniky zkracování zpětné vazby
 - Vývoj řízený testy (Test-driven development, TDD)
 - Párové programování
 - Debugování s gumovou kachničkou
 - Společná odpovědnost za kód
 - Refaktorizace bez nutnosti svolení
 - Krátké iterace v řádu hodin až dnů
 - Zákazník je na pracovišti jako člen týmu vývojářů



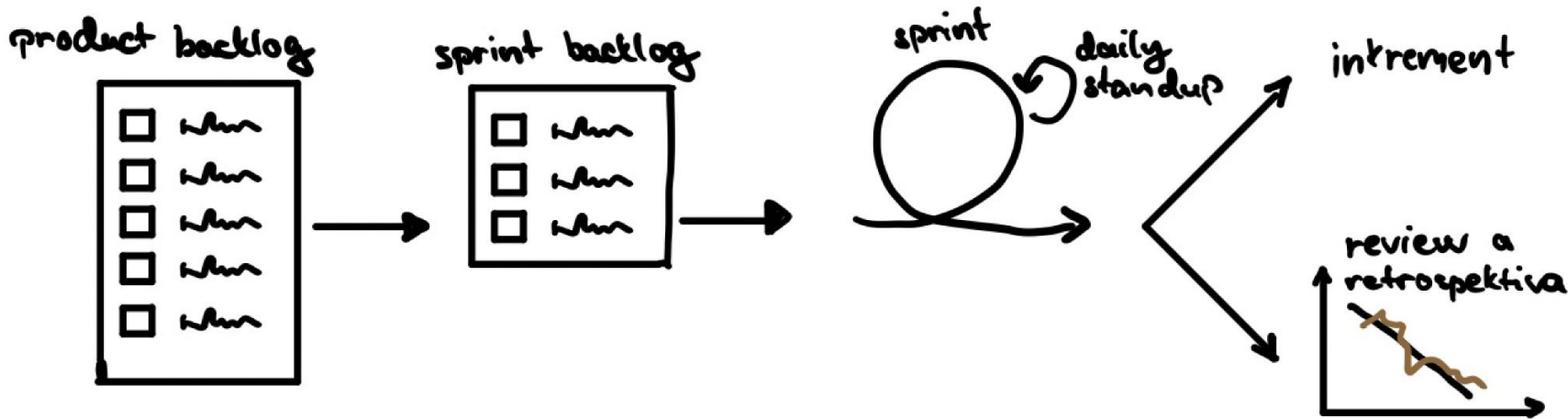
1.6. Mlýn (Scrum)

- Role

- **Vlastník produktu** - zastupuje zadavatele, určuje priority požadavků, řeší financování
- **Scrum master** - odstraňuje překážky vývojářům, stíní, organizuje a vede vývojáře
- **Vývojáři** - tým vývojářů je typicky složen z 5 až 7 lidí (max 2 pizzy na nasycení)

- Artefakty

- **Produktový zásobník** - všechny požadavky zainteresovaných osob
- **Zásobník sprintu** - požadavky na aktuální sprint
- **Softwarový přírůstek** - předchozí vydaná verze obohacená o vyvinuté požadavky sprintu



1.7. Metriky postupu vývoje softwaru

- Další užitečné artefakty Scrumu pro měření efektivity vývoje
 - Graf vyhoření (Burndown chart)
 - liniový graf ukazující zbývající práce v čase
 - Definice hotového (Definition of Done, DoD)
 - definice podmínky, které musí být splněny, aby byl požadavek považován za úspěšně implementovaný
 - Okamžitá rychlost (velocity)
 - graf porovnání předchozího a aktuálního sprintu pro pochopení kapacit pro vývoj (ale zneužitelný)
 - Špička (spike)
 - časově ohraničený úsek, při kterém tým může zkoumat technologie a přijít s technologický důležitým prototypem



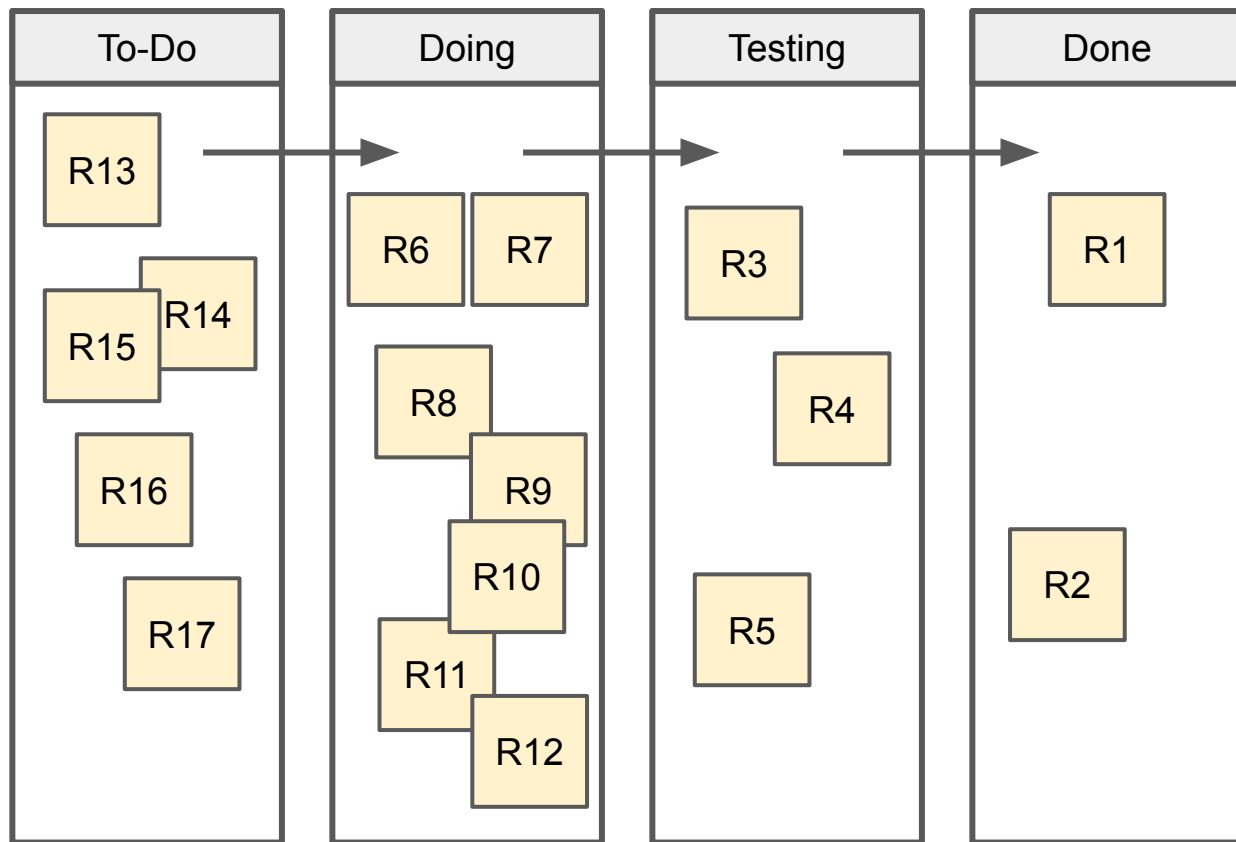
1.8. Organizace vývojových požadavků (Kanban tabule)

Uživatelský příběh

Já (role): Učitel

Chci: Zapsat v záložce zápočet studentům úspěšné splnění kurzu.

Abych: Mohl zajistit roli student úspěšné přiřazení zasloužených kreditů.



1. Shrnutí

- Existují různé způsoby jak vyvíjet software, avšak všechny obsahují základní aktivity - analýza, implementace, testování, evoluce.
- Metodiky vývoje se rozdělují podle toho, zda se vyvíjí v iteracích nebo sekvenčně, a podle toho, zda se ztotožňujeme s agilním manifestem.
- Mezi nejpopulárnější agilní metodiky vývoje patří Scrum, který nejčastěji v dvoutýdenních iteracích (sprinty) vyvíjí inkrement softwaru za asistence svého ochránce (scrum master) a zástupce klienta (product owner).
- Požadavky klienta (nejčastěji ve formě uživatelských příběhů) mají svůj životní cyklus a v iterativních metodikách se často přesouvají po Kanban tabuli.

1. Výzkumné náměty

1. **Porovnání efektivity metodik vývoje softwaru** (teoretická práce)

- Provedte systematickou rešerši vědeckých článků, které porovnávají efektivitu metodik vývoje softwaru. Využijte k tomu metodiku PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses)

2. **Smart Kanban tabule** (praktická práce)

- Vytvořte open-source aplikaci, která slouží jako interaktivní Kanban tabule a automatizovaně měří a zobrazuje různé metriky vývoje kódu (burndown, velocity a další, které naleznete).

3. **Metodika vývoje softwaru pro studentské týmy** (výzkumná práce)

- Vymyslete metodiku vývoje softwaru, která je vhodná pro tvorbu projektů na naší univerzitě, kde jsou týmy většinou v počtu 3 studentů. Ověřte efektivitu metodiky pomocí A/B testu na týmech, které aktuálně nějaký projekt vyvíjí.

4. **GPT v párovém programování a debuggování s gumovou kachničkou** (výzkumná práce)

- Provedte průzkum efektivity párového programování s ChatGPT a debuggování s gumovou kachničkou s ChatGPT (kachnička bude GPT) a ověřte na vzorku studentů pomocí A/B testu.

1. Cvičení - Párové programování s ChatGPT

- Zahrajte si s ChatGPT na párové programování
 - Vytvořte mu pomocí chytrého promptu roli v párovém programování a střídejte se s ním.
 - Driver - programuje
 - Navigator - radí při vývoji
 - Naprogramujte si tímto způsobem nějaký jednoduchý organizátor úkolů a na konci hodiny budeme diskutovat o efektivitě a přínosech.

Seminář 2

Inženýrství požadavků



2.1. Proces inženýrství požadavků

- Inženýrství požadavků
 - Metodika (až téměř synonymum) pro fázi analýzy požadavků na systém z obecného model
- Aktivita a artefakty procesu inženýrství požadavků:
 - **studie proveditelnosti** (artefakt: report o proveditelnosti)
 - **sběr požadavků** (artefakt: hrubý seznam požadavků)
 - **analýza požadavků** (artefakt: očištěný seznam požadavků)
 - **specifikace požadavků** (artefakt: dokumentace specifikace požadavků - uživatelské a systémové)
 - **validace požadavků** (artefakt: odsouhlasená dokumentace specifikace požadavků)
- Pozor na zmatečnou terminologii
 - První velká fáze v obecném modelu je Analýza požadavků
 - Inženýrství požadavků je metodika, jak provádět tuto velkou fázi
 - Inženýrství požadavků obsahuje dílčí aktivitu Analýza požadavků, která představuje něco jiného

2.2. Analýza proveditelnosti

- Potřebujeme získat následující informace pro rozhodnutí o případném přijetí projektu:
 - Je pro nás projekt zajímavý? (finančně, pověst)
 - Máme k dispozici všechny potřebné zdroje? (čas, vývojáři, technologie)
 - Máme know-how pro řešení klientova problému?
 - Chceme takový projekt vyvíjet? (třeba pro nás není zajímavý)

2.3. Sběr požadavků

- Tuto fázi procesu (requirements gathering) lze rozdělit na dvě dílčí aktivity:
 - **Počátek sběru (requirements inception)** - počáteční sběr
 - **Vyjednávání o požadavcích (requirements elicitation)** - kontinuální iterativní sběr
- Pro zvýšení efektivity sběru musí IT analytik provést následující úkony:
 - seznámit se s problémovou doménou
 - seznámit se s problémy, které má software uživatelům vyřešit
 - seznámit se s omezeními, které jsou na software kladeny
- Zdroje pro získání požadavků:
 - **uživatelé softwaru** - ten, kdo software bude při práci používat
 - **zajímavované osoby** - všichni ostatní, kdo mají na vývoji softwaru zájem (typicky vedení)
 - **doménový experti** - najatí experti, kteří vědí o kontextu a softwaru daného typu nejvíce
 - **existující systémy** - analýza aktuálně používaných SW nebo konkurenčních řešení
 - **dokumenty** - dokumenty podniku mohou říci analytikovi hodně o požadavcích jak funkčních (dokumentace podnikových procesů), tak legislativních/standardizačních.

2.4. Nástroje pro sběr požadavků

- Existuje velké množství nástrojů a vyjednávacích technik pro sběr požadavků
 - **rozhovor (interview)** - popovídám si s osobou z očí do očí a oba mi vypráví své představy, já se doptávám na nejasnosti.
 - **strukturovaný rozhovor** - to stejné jako interview, ale mám přesně zadané otázky, na které se ptám a které pomohou architektovi software navrhnout.
 - **dotazník** - zašlu strukturovaný dotazník všem zainteresovaným osobám a ty mi jej vrátí s odpověďmi, ze kterých získám požadavky (asi nejhorší metoda).
 - **bouře mozků (brainstorming)** - pozveme více osob na schůzku, kde si hromadně vyměňujeme nápady.
 - **pozorování uživatelů (task observation, ethnography)** - sleduji uživatele používat aktuální software nebo pracovat a zapisuju si, jaký je postup a vymyslím případně jak práci zefektivnit.
 - **modelování případů užití** - pomocná technika, kde již získané požadavky vymodeluju diagramem a následně se bavíme s osobou o tom, co lze přidat.
 - **prototypování** - pomocná technika, ve které vytvořím statický nebo interaktivní prototyp (třeba ve Figma), díky kterému s klientem přijdeme na další požadavky nebo upravíme stávající.

2.5. Typy požadavků

- Dvě formy požadavků
 - **Uživatelské** - napsáno slovy klienta, klient musí těmto požadavkům rozumět pro úpravy
 - **Systémové** - napsáno pro vývojáře, využívá se žargonu IT oboru
- Dva typy požadavků
 - **Funkční** - jaké funkcionality musí software vykazovat při interakci s ním
 - **Mimofunkční** - požadavky, které jsou kladené na software a nesouvisí s funkcionalitou
- Typy mimofunkčních požadavků
 - **Produktové** - požadavky na chování produktu z dependabilního hlediska (spolehlivost, bezpečnost, zabezpečení, udržitelnost aj.)
 - **Organizační** - požadavky vycházející z vnitřního prostředí podniku, tedy politiky, standardy a procesy podniku
 - **Vnější** - požadavky vycházející z legislativy nebo propojitelnosti s jinými produkty

2.6. Analýza požadavků

- Analýzu provádíme za účelem očištění požadavků s klientem, abychom získali co nejkvalitnější požadavky. Kvalitní požadavky:
 - **jsou jasné** - klient chápe, co se požadavkem myslí z podnikového hlediska
 - **jsou pochopitelné** - programátor chápe, co se požadavkem myslí z program. hlediska
 - **jsou kompletní** - požadavek má všechny potřebné informace pro implementaci
 - **jsou specifické** - požadavek je dostatečně detailní pro implementaci
 - **jsou proveditelné** - požadavek je reálné implementovat
 - **jsou ověřitelné** - jsme schopni ověřit úspěšnou implementaci
 - **jsou kvantifikovatelné** - pro mimofunkční požadavky, musíme být schopni číselné změřit
 - **jsou nekonfliktní** - není v konfliktu s jiným požadavkem (nejen výlučně, ale i částečně)
- Proces čištění požadavků provádíme pomocí
 - **logické analýzy** - povinné minimum, představím si požadavek v hlavě a pracuju s ním
 - **formálního hodnocení** - zkoumání výše uvedených vlastností kvalitních požadavků
 - **prototypování** - zkoumání požadavků na grafickém nebo i programovém prototypu

2.7. Specifikace požadavků

- Proces transformace požadavků zainteresovaných osob ve formální podobě do artefaktu Dokumentace specifikace požadavků
 - **uživatelské** - napsáno slovy klienta
 - **systémové** - napsáno na základě slov klienta pro vývojáře
- Z dokumentace by mělo být schopno odpovědět na následující otázky:
 - jaký je účel systému, který chce klient vyvinout?
 - jaké problémy, které aktuálně klient má, bude systém řešit?
 - jak systém vyřeší tyto problémy?
 - jak bude systém využíván na denní bázi?
 - jak problémy s výkonem systému ovlivní používání systému na denní bázi?
- Pozor na Jeden zdroj pravdy (Single source of truth)
 - Může se snadno stát, že různí lidé mají různé verze i různé formy (kanban vs textový dokument) toho, co se má formálně vyvinout a způsobuje to chaos.
 - Zejména u agilních metodik (tam sice dokumentace tolik důležitá, ale také ji vytváříme).

2.8. Modely specifikací požadavků

- Požadavky lze formalizovat do následujících modelů
 - **Přirozený jazyk (jen mu dáme nějakou korekturu)**
 - všichni mu rozumí, avšak může být vágní při interpretaci vývojáři
 - **Přirozený jazyk s formální strukturou**
 - role, vstupy, výstupy, podmínky, proč to role chce?
 - **Pseudokód**
 - velice dobře interpretovatelné vývojářem, nikoliv klientem
 - **Grafický zápis**
 - kompromis pro vývojáře i klienty, nejčastěji se využívá grafický modelovací jazyk UML
 - **Matematický zápis**
 - vhodné pouze u bezpečnostně kritických systémů

2.9. Správa požadavků

- Požadavky mají vlastní životní cyklus
 - Zejména v agilních metodikách může docházet často k jejich změnám
- Dva způsoby řízení životního cyklu požadavků
 - Psát změny požadavků do dokumentace specifikace požadavků
 - Využít specializovaný software pro správu požadavků
- Funkcionality systému pro správu požadavků (**červené body často chybí**)
 - CRUD operace nad požadavky (vytvoření, prohlížení, úprava, mazání)
 - Prioritizovat požadavky
 - Přidělovat vývojářů požadavky k implementaci
 - Zohlednit role a důležitost zainteresovaných osob
 - **Propojit se specifikacemi požadavků v zadávací a systémové dokumentaci (trasovatelnost)**
 - **Vypisovat historické a statistické informace o životním cyklu požadavků**

2. Shrnutí

- Analýza požadavků je fáze vývoje softwaru, při které získáváme pomocí procesu Inženýrství požadavků kvalitní formální zápis požadavků k vývoji.
- Jedná se o náročnou fázi, při které IT analytik sbírá požadavky a vyjednává o nich s klientem pomocí různých nástrojů (dotazníky, rozhovor, prototypy). Vzájemné nepochopení může vést k neúspěchu projektu.
- IT analytik by měl být expertem na mezilidskou komunikaci, vyjednávání (musí umět říct ne) a musí mít znalost problemové domény.
- Požadavky je možné reprezentovat pomocí různých modelů. Aktuálně existuje problém s trasovatelností životního cyklu požadavků .

2. Výzkumné náměty

1. **Porovnání efektivity technik a nástrojů při vyjednávání požadavků** (teoretická práce)
 - Proveďte systematickou rešerši vědeckých článků, které porovnávají efektivitu technik a nástrojů pro sběr požadavků. Využijte k tomu metodiku PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses)
2. **Software pro trasovatelnost požadavků** (praktická práce)
 - Vytvořte open-source aplikaci, která slouží jako zapisovač požadavků. Aplikace bude ukládat historii změn požadavků a bude vytvářet různorodé reporty o životním cyklu požadavku. Při změně požadavku automatizovaně upraví dotčené artefakty, aby nedošlo k narušení jednotného zdroje pravdy
3. **Upravená metodika inženýrství požadavků pro studentské týmy** (výzkumná práce)
 - Upravte metodiku inženýrství požadavků tak, aby byla vhodná pro tvorbu projektů na naší univerzitě, kde jsou týmy většinou v počtu 3 studentů. Ověřte efektivitu metodiky pomocí A/B testu na týmech, které aktuálně nějaký projekt vyvíjí.
4. **Jazykový model jako agent pro vyjednávání o požadavcích** (výzkumná práce)
 - Proveďte průzkum efektivity jazykových modelů jako asistentů pro vyjednávání o požadavcích. Ověřte na vzorku studentů pomocí A/B testu.

2. Cvičení - Umí GPT3.5 trasovat požadavky?

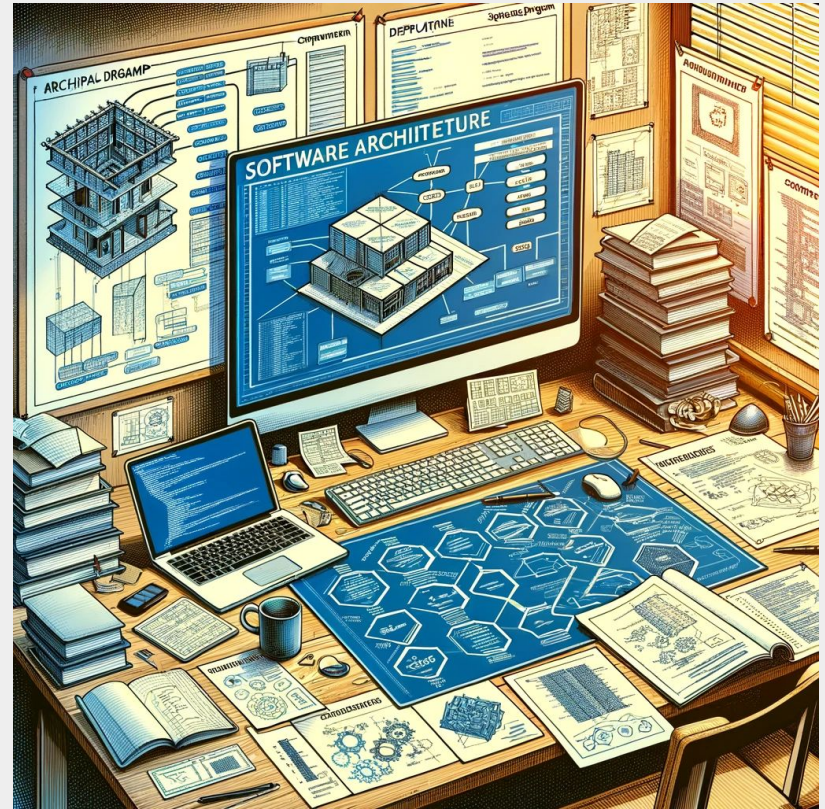
1. Zadejte do ChatGPT požadavky v přirozeném jazyce na software, který má sloužit jako kalkulačka se základními operacemi.
2. Nechte ho následně vygenerovat Diagram případů užití v jazyce plantUML a zobrazte si ho ve specializovaném webovém softwaru (najděte). Zkontrolujte správnost.
3. Dále ho nechte k požadavkům vygenerovat uživatelské příběhy. Zkontrolujte správnost.
4. Nechte ho k dílčím požadavkům vygenerovat kód v jazyce Python s jednotkovými testy a ověřte funkčnost.
5. Vymyslete si nějaký změnový požadavek na stávající funkcionalitu a nechte ho opravit Diagram případů užití, uživatelské příběhy a spustitelný kód. Povedlo se mu správně trasovat změnový požadavek napříč všemi třemi artefakty?
6. Vytvořte si nové okno v chatu a testujte stabilitu GPT v tomto experimentu.

2. Cvičení - Umí GPT3.5 radit se sběrem požadavků?

1. Udělejte dvojice. Jeden z dvojice bude v roli klienta a druhý v roli analytika.
2. Zadejte do ChatGPT požadavek na asistenci při sběru požadavků od klienta pro Vámi vybraný software. Dejte mu takový prompt, aby Vám Chat radil s tím, na jaké otázky se zeptat nebo případně doptat na nejasné požadavky.
3. Sepište si výsledné sesbírané požadavky a dejte je Chatu k finální analýze.
4. Nechte Chat vygenerovat požadavky ve Vámi vybrané formální podobě.
5. Posudte, jak moc byl Chat efektivní při pomoci s vyjednáváním o požadavcích.

Seminář 3

Softwarová architektura



3.1. Fáze implementace

- Po odsouhlasení dokumentace aplikace začíná návrhová fáze, které se účastní několik rolí (v praxi zastupuje jedna role více ze zmíněných rolí)
 - **Systémový inženýr** - navrhuje holistické propojení celého systému za účelem cíleného emergentního chování; zabývá se lidmi, hardwarem, vyvíjeným softwarem a na něj napojenými aplikacemi a modeluje systém pomocí sysML nebo kontextových diagramů
 - **Vedoucí technické stránky vývoje** - volí technologický zásobník pro navržený systém
 - **Softwarový architekt** - navrhuje komponenty a jejich propojení, ze kterých se vyvíjený software skládá, často ve formě UML diagramů
 - **Návrhář komponent** - navrhuje vnitřek komponent (atributy, operace), detalizuje UML diagramy, zejména diagram třídy (class diagram)
 - **Programátor (junior, medior, senior)** - implementuje vnitřek komponent
 - **Návrhář uživatelského rozhraní (UI)** - navrhuje vzhled rozhraní aplikace pro uživatele, vytváří k tomu drátěné modely (wireframes) a jednoduché grafické prototypy (mockups)
 - **Návrhář uživatelského zážitku (UX)** - optimalizuje průchod aplikací pro uživatele, používá k tomu vizuální prostředky jako uživatelské dobrodružství (user-journey) a A/B testování
 - **Návrhář interakčního zážitku (IdX)** - optimalizuje zpětnou vazbu (chyba, úspěch, čekání) z aplikace uživateli pro jeho optimální informovanost

3.2. Návrh architektury

- Softwarové architektury mají své archetypy (obecné vzory)
 - **Vrstevnatá architektura** - dekompozice programu do N vrstev, *př.: TCP/IP*
 - **Klient-server** - hlavní komponenta (server) poskytuje služby ostatním (klient), *př.: Netflix*
 - **Hlavní-vedlejší (Master-slave)** - hlavní komponenta (master) přiřazuje práci ostatním (slave), *př.: paralelní a distribuované vědecké výpočty*
 - **Roury-filtry (Pipes-filters)** - postupné zpracování dat skrze sekvenci aktivit (filtry), *př.: ETL transformace v datové analýze*
 - **Zprostředkovatel (Broker)** - zprostředkovatel komunikuje se servery in proxy za klienty a vyjednává s nimi nastavení služby, *př.: RabbitMQ*
 - **Peer-to-peer** - všichni jsou zároveň servery i klienty a vzájemně žádají o služby, *př.: Torrenty*
 - **Událost-sběrnice (Event-bus)** - zdroje událostí zasílají zprávy sběrnici pro naslouchače, které se registrovaly pro náslechy, *př.: Mobilní aplikace*
 - **Model-view-controller** - model obsahuje data, controller logiku a view vizualizaci, *př.: web*
 - **Tabule (blackboard)** - tabule je globální paměťový prostor s problémem, zraje znalostí se hlásí o slovo pro aktualizaci stavu problému, řadič je řídí (jako ve škole), *př.: AI, NLP*
 - **Tlumočník (interpreter)** - čtení instrukcí řádek po řádku a jejich přetlumočení do příkazů, typicky se rozkládají do stromu příkazů, *př.: interpretry programovacích a dotazovacích jazyků*

3.3. Návrh komponent

- Komponenta je prvek systému, který je odpovědný za nějakou izolovanou činnosti (funkce, třída, kolekce tříd až skoro podsystémy). Nejčastěji:
 - **Procedurální programování** - komponentou je modul, atributy jsou globální a lokální proměnné, operace jsou funkce a procedury
 - **Objektově-orientované programování** - komponentou je třída, atribut je členská proměnná/datový člen a operace jsou metody
 - **Funkcionální programování** - komponentou je modul, atributy neexistují (v čistě funkcionálním přístupu neexistuje stav systému), operace jsou funkce
- V OOP paradigmatu nejčastěji navrhujeme komponenty pomocí UML jazyka
 - **Diagramy struktury** - pro modelování struktury komponent a systému
 - Diagramy komponent, tříd, objektů, složené struktury, balíčků, nasazení, profilů
 - **Diagramy chování** - pro modelování chování komponent a systému
 - Diagramy případů užití, aktivit, stavů
 - **Diagramy interakce** - pro modelování datového toku a řízení uvnitř komponent a systému
 - Diagramy sekvencí, komunikace, časování, přehledu interakcí

3.4. Principy návrhu komponent

- Existují pravidla, která je vhodné při návrhu dodržet
 - **Modularizace (modularization)**
 - Vyvíjený software je nutné rozdělit do dílčích komponent s oddělenou odpovědností.
 - **Abstrakce (abstraction)**
 - Při návrhu komponent se zaměřuji pouze na podstatné atributy a operace.
 - **Zapouzdření (encapsulation)**
 - Komponenty by měly skrýt implementační detaily pomocí vlastností a rozhraní.
 - **Propojení (coupling)**
 - Komponenty by měly být na sobě co nejméně závislé, změna jedné neovlivní jiné
 - **Soudržnost (cohesion)**
 - Komponenty by měly spolu pracovat skrze rozhraní bez co nejmenšího množství lepidla
 - **Oddělení rozhraní od implementace (separation of interface and implementation)**
 - Komponenty spolupracují pouze přes veřejné rozhraní
 - **Celistvost (completeness)**
 - Rozhraní komponenty má podle svého účelu všechny typicky žádané služby

3.5. Návrhové vzory

- Při navrhování komponent systému pomocí tříd narazíte na spousty problémů, které už někdo jiný řešil (také archetypy, ale menší)
 - Vzory pro tvorbu
 - poskytují mechanismy efektivní a udržitelné tvorby komponent
 - jedináček, tovární metoda, abstraktní továrna, stavitel, prototyp
 - Vzory pro strukturu
 - poskytují mechanismy strukturování vnitřku komponent na efektivnější komponentu
 - fasáda, adaptér, prostředník, dekorátor, muší váha, most, kompozit
 - Vzory pro chování
 - poskytují mechanismy, které dodávají komponentám dodatečné flexibilní chování
 - řetěz odpovědnosti, příkaz, iterátor, prostředník, memento, pozorovatel, stav, strategie, šablonová metoda, návštěvník

3.6. Přístupy k vývoji komponent

- Ať použijete jakoukoliv datovou strukturu pro reprezentaci komponent, musíte navrhnout vyšší komponentní celky pomocí nějakého přístupu.
- Top-Down
 - Představím si chování komponenty a začnu postupně rozkládat komponentu na dílčí. Postupným voláním dílčích komponent dojdou k výslednému touženému chování.
- Bottom-Up
 - Začnu budovat atomární komponenty (funkce, třídy), které poté spojují do větších celků. Zde hrozí porušení principu YAGNI.
- Dopředný a zpětný návrh
 - V praxi vývojáři nenavrhují kód příliš předem (dopředný návrh)
 - Komponenty systému se spíše hledají v průběhu implementace a dokumentuje se architektura aplikace po vývoji (zpětný návrh)

3.7. Základní poučky vývoje komponent

- KISS (Keep It Simple and Stupid)
 - Programujme komponenty tak jednoduché, ať je pochopí každý a může je po nás upravovat.
- DRY (Don't Repeat Yourself)
 - Potřebujeme-li opakování chování, abstrahujme ho do vyšších celků (funkce, třída, modul).
- YAGNI (You Ain't Gonna Need It)
 - Neprogramujte jakékoliv komponenty, které nejsou na základě požadavků zapotřebí.
- AHA (Avoid Hasty Abstraction)
 - Nepoužívejte přílišnou generalizaci pokud nepotřebujeme vytvářet příbuzné potomků
- SOLID (pro třídy, ale občas i pro obecné komponenty)
 - **S = Single responsibility princip** - třída má být odpovědná za jeden typ úkonů
 - **O = Open/Closed princip** - třída má být otevřena pro rozšíření, ale uzavřena pro přepsání
 - **L = Liskovské substituční princip** - Typ může být nahrazen svým podtypem.
 - **I = Interface segregation princip** - Pokud má třída příliš složité rozhraní, optimalizujme ho buď do více metod s krátkým rozhraním nebo/a využijme polymorfismus
 - **D = Dependency inversion princip** - Mezitřídní komunikace nemá záviset na implementaci. Má záviset na abstrakci.

3.8. Návrh uživatelského rozhraní (UI, User Interface)

- **Nástroje pro návrh a optimalizaci uživatelského rozhraní**
 - **Drátěný model (Wireframe)** - rychlý hrubý náskres rozložení prvků
 - **Maketa (Mockup)** - realistický grafický návrh, který připomíná finální produkt
 - **Prototyp (Prototype)** - interaktivní simulace finálního produktu
- **Principy návrhu uživatelského rozhraní**
 - **Jasnost (Clarity)** - logická organizace prvků podle konceptů z tvarové (gestalt) psychologie
 - **Konzistence (Consistency)** - chování aplikace se nemění, je jednotné a předvídatelné
 - **Zotavitelnost (Recovery)** - uživatel by měl snadno navrátit svá rozhodnutí a aplikace by se měla vhodně zotavit z chyb
 - **Pohodlnost (Comfort)** - uživatel by měl snadno chápat používání aplikace (včetně obsahu)
 - **Reakceschopnost (Responsiveness)** - poskytování zpětné vazby o stavu aplikace
 - **Přístupnost (Accessibility)** - inkluzivní techniky pro lidi s handicapem
 - **Ovladatelnost (User control)** - uživatel by měl být schopen si upravit UI podle svých preferencí

3.9. Uživatelský zážitek (UX, User eXperience)

- Nástroje pro optimalizaci uživatelského zážitku UX (User eXperience)
 - **Mapa empatie** - modeluje znalosti o konkrétním uživateli
 - **Mapa dobrodružství** - modeluje, čím konkrétní uživatel nebo role prochází během úkolu
 - **Mapa zážitku** - více obecné než dobrodružství, pracuje s obecnou rolí
 - **Šablona služby** - odvrácená strana mapy dobrodružství, ukazuje, co se děje na pozadí

3. Shrnutí

- Návrh softwaru zahrnuje spousty dílčích na sebe navazujících návrhářských aktivit od návrhu architektury po návrh samotných komponent a rozhraní
- Pro návrhové potřeby existují archetypy ve formě architektonických vzorů pro softwarovou architekturu a návrhových vzorů pro návrh komponent.
- Systém se skládá z podsystémů, které se skládají z komponent. Komponenty se skládají z jiných komponent až na atomické jako jsou funkce a třídy.
- Při návrhu a vývoji komponent dodržujeme několik zásadních pouček, jejichž cílem je zkvalitnit výslednou kódovou bázi.
- Progresivní obor je optimalizace UI a UX, pro které stále vznikají nástroje.

3. Výzkumné náměty

-

3. Cvičení - Rychlý návrh aplikace

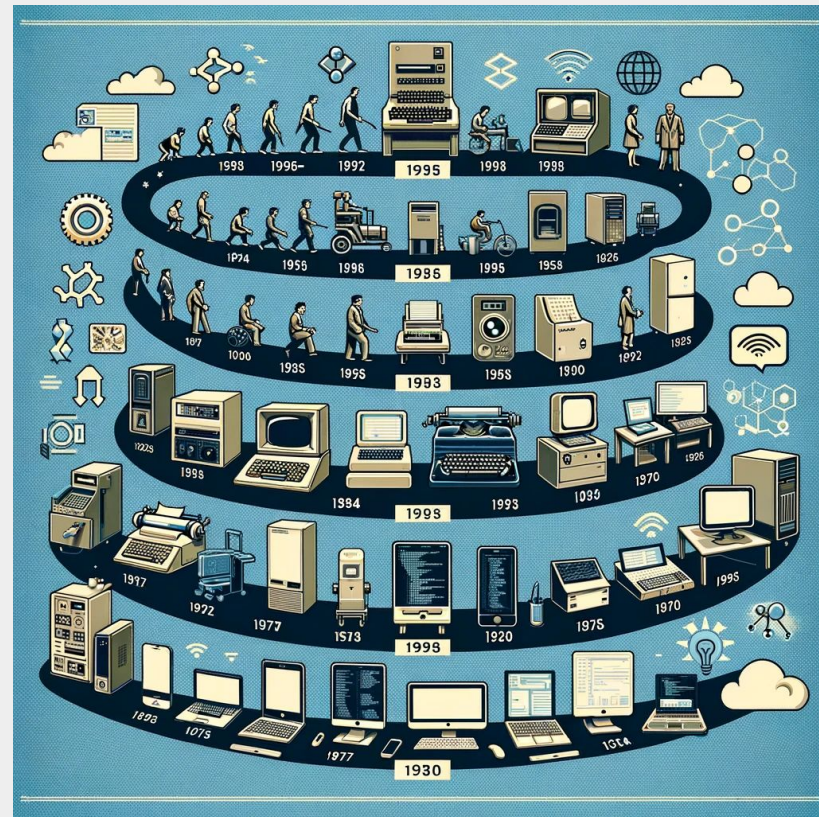
- Udělejte dvojice a navrhnete drátěný model aplikace s následujícími funkčními požadavky (Zoo Quest).:
 - a. Uživatel se přihlásí do aplikace, jinak ho aplikace nikam nepustí.
 - b. Po přihlášení vidí svou kameru s návodným čtverečkem pro skenování QR kódu. QR kódy se skenují v Ústecké zoo u výběhu zvířat. Pokud by naskenoval jiný QR kód, tak aplikace vyhodí ve stejném okně krátké modální okno s chybovou hláškou. Toto úvodní okno budeme nazývat QR-Hunt.
 - c. Další okno (může přepínat buď swipováním zprava-doleva nebo v dolní navigační liště klikem na položku menu) je okno s názvem Pokézoo, kde vidí modely různých zvířat, které pomocí QR čtečky oskenoval. Při kliku na ikonu zvířete se otevře okno s detailním 3D modelem zvířete a informacemi o něm.
 - d. Předposledním oknem je okno s názvem Game, kde nalezne různé výzvy pro dnešní týden. Výzvy mají obtížnost a progress bar, který říká v procentech na kolik je s úkolem uživatel hotov.
 - e. Poslední okno je Profil, kde nalezne základní informace o svém účtě a obdržené odznaky (také jim říkáme badges, trophies, achievements).

3. Cvičení - Optimalizace uživatelského zážitku

- Vytvořte si pomocí Mapy empatie alespoň 3 různorodé osoby. Můžete klidně použít ChatGPT, aby Vám pomohl s charakteristikami.
- Poté využijte tyto osoby pro vytvoření Map dobrodružství. Přemýšlejte, jak by osoba reagovala a na základě toho dejte programátorům doporučení na zlepšení UI, Ix nebo samotné funkcionality.

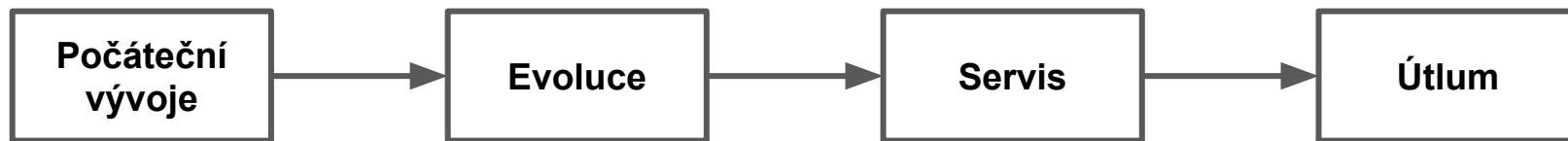
Seminář 8

Evoluce softwaru



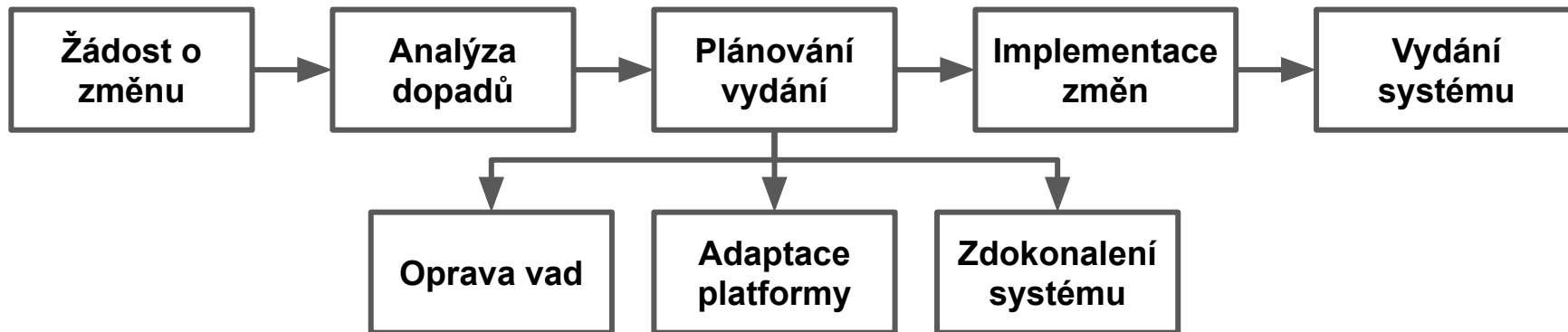
8.1. Fáze evoluce

- lorem



8.2. Proces evoluce softwaru

- lorem



8.3. Lehmanovy zákony

8.4. Refactoring

8.5. Fowlerovy zápachy

8.6. RefaktORIZAČNÍ techniky

8.7. Údržba starých systémů

8.8. Reinženýring

8.9. Výpočet ceny údržby

8. Shrnutí

8. Výzkumné náměty